Nyay Saathi: an AI legal helper for everyday problems in India

Kuku AI Generalist assignment, **Problem 2: Build an AI Lawyer** · Karan · 230121033 · Engineering Physics · Code, outputs and demo video linked in the submission form

1. What I built, and why this scope

A chat assistant that takes a messy story and helps the person act: it **builds the case** (what happened, when, how much, what proof), **explains where they stand and their options** in plain language, **drafts a legal notice**, and lets them **practise the argument** with the assistant playing the other side. It also **explains a legal notice the person has *received*** and drafts a reply template with review instructions, gives an **evidence checklist** and a **reminder calendar (.ics)**, and lets them **download / print** their case file. The interface and print/download controls can be selected in nine Indian languages; optional Gemini translation localizes grounded replies and documents. The app masks recognized personal identifiers and has a **one-shot mode** that turns any pasted story into a full report.

Scope: three problems, in depth, plus received notices: (1) a landlord keeping the security deposit, (2) a refund / defective product or service, (3) unpaid salary. I chose them because they are the brief's own examples, are everyday and money-sized, are decided mostly on documents (so a chat can really help), and each has a clear route (civil court, Consumer Commission, Labour Commissioner). Family, criminal, property-title, tax, cyber and builder/RERA matters are **deliberately out of scope**: the assistant says so and points to a lawyer / free legal aid (NALSA 15100).

2. Assumptions (the brief was silent)

- User is an ordinary adult on a phone describing a money claim and no lawyer yet. English/Hindi and Hinglish are handled by the core workflows; an optional Gemini connection can translate supported regional-language text. "Help them act" means **information + paperwork + rehearsal**, not filing; a real lawyer is always recommended before filing.
- Free tier only. With no API key available, the default build runs **offline at zero cost**; an LLM is an optional plug-in.
- Legal content was drafted from my own knowledge, then checked on 8 Nov 2026 against **India Code, Gazette notifications and official department sources** where available; some practice guidance uses explainers. *This is not a lawyer's review.* The check **found two outdated entries**: the Payment of Wages Act and the Industrial Disputes Act were replaced on 21 Nov 2025 by the Code on Wages and the Industrial Relations Code. These were corrected, and each entry stores its verification status. Limits and state rules can change, so answers ask users to confirm them.

3. How it works: what is AI, what is code, where a human stays

Step	Done by	Detail
Language + problem type	Code (keyword lexicon, EN/HI/Hinglish)	Nine-language interface selector; selected language follows grounded replies when translation is available. Unclear or out-of-scope → asks or refers; emergencies (arrest, violence) → 112 / NALSA.
Case building	Code (regex slot-filling)	Asks ≤4 questions: when, amount, proof, asked-in-writing. If several amounts appear it asks which is owed (a bug found in testing).
Legal content	**Closed knowledge base** (16 vetted entries, EN+HI)	The only place law lives. Every answer lists its sources with a link and confidence.
Deadlines, forum	Code	Limitation check (3 yrs deposit / 2 yrs consumer) with near/expired warnings; Consumer Commission limits use value paid as consideration, which may differ from the refund due.
Notice drafting	Templates	Fills only facts the user gave; unknowns stay as `[BRACKETS]`; never invents facts.
Notice you received	Code + template	Reads sender, amount, deadline, threats (court / police / cheque / bank loan); flags urgent cases; drafts a reply with `[CHOOSE ONE]` for dispute / part / pay and a reminder to edit prompts and get legal review. Does not imply the recipient can stop the other party's next steps.
Privacy	Code	Detects and masks IDs before anything is routed, stored, exported or sent to an LLM; warns the user. Failure log is opt-in and redacted.
Evidence checklist, calendar, export	Code	Proof ticked vs missing; approximate dates (send notice, escalate, time-limit review) as .ics; summary, notice, checklist and received-notice reply as print-to-PDF or .txt, with print controls localized in all nine interface languages.
Practice mode	Scripted opponent + rule-based coaching	4 realistic objections per topic; tip after each reply; checklist at the end.
Other languages, plain wording	**LLM (optional, swappable)**	Gemini / OpenAI-compatible / Anthropic via env vars. Translates in, simplifies out; output must pass the guard.
Human in the loop	Lawyer + reviewer	Lawyer reviews the knowledge base before launch and quarterly; assistant routes to a real lawyer on big amounts, near time limits, thin proof.

4. How it avoids inventing laws, sections and cases

Five layers, from "can't" to "caught": **(a)** the model never writes legal content (retrieval-free closed list, so there is nothing to confabulate); **(b)** an **output guard** scans every message and blocks any Act, Section, case-style citation (*X v. Y*, AIR, SCC) or

new number not in the knowledge base, replacing it with "check with a lawyer"; **(c)** case-law requests are refused ("AI tools often invent them"); **(d)** prompt-injection and "guarantee I'll win" are handled; **(e)** sections are cited only where confidence is high, otherwise just the Act name.

5. How I know it works, and where it fails

Check	Result
26 scripted conversations I wrote (EN, Hindi, Hinglish; happy paths, time-barred, large claim, divorce, arrest, case-law request, injection, vague, gibberish, multiple amounts, received notices, checklists)	**26/26 pass** after fixes; first run was 11/15 (see below). Raw transcripts in `outputs/transcripts/`
49 extra checks: exports (including all nine print-control locales), calendar, privacy, date parsing, selected-language replies, safer notice-reply prompts, LLM/Gemini configuration and Punjabi translation, 5 one-shot reports	**49/49 pass**; all shown in `outputs/dashboard.html`
Problem-type detection, labeled sets (deposit / refund / salary / out-of-scope; EN, Hindi, Hinglish)	First run **83%** correct, 2 wrong. After tuning on it: 98% (optimistic). On a **frozen set of 30 fresh messages: old rules 73% → upgraded rules 80%, 0 wrong** (phrase patterns, "ask if two types are close" rule, word-boundary fix). Earlier held-out set: 58% → 71%, wrong advice 8% → 4%.
English vs Hindi parity: same 8 stories told in both languages; speed	**100%** of 72 fields match (first run 90%: 5 diffs were my test's bug, 2 real: Hindi "इनवॉइस" not read as invoice). Latest offline run measured p95 **1.75 ms**/message.
Guard attack: a fake LLM that hallucinates a fake Act, wrong section, fake case, AIR citation, changed number, section with no Act	**7/7 blocked**; an honest rewrite is accepted; end-to-end with a hallucinating model the user still gets the grounded text (`outputs/results.json`)

Bugs my tests found (all fixed): keyword "lease" matched inside "p*lease*" (substring matching), so every polite message leaned to *deposit*; the amount parser swallowed "2 months**,** Rs 70000" (a comma read as a number); the guard wrongly blocked a correct answer because of a long official Rules title; "80000 sal*ary*" was parsed as "80000 *saal*" (years); an unclassifiable message made it repeat one question forever; the first of several amounts was silently taken; the "will I win?" warning was lost when asked in an earlier turn; running my own sample stories showed the date parser used the *tenancy start* date instead of the move-out date and could not read Hindi months, which would give a false "time-barred" warning.

Where it still fails (honest list; raw in `outputs/probes.md`):

- **Keyword rules generalise only partly (measured):** 98% on messages I tuned against vs **71-80% on fresh ones**, 0-4% wrong advice; unplaceable messages get a question, and many out-of-scope topics are "asked" not clearly "referred". Mitigations: the first reply names the detected problem and the user can correct it in one message ("wrong, it's a deposit problem"); amounts/dates can be corrected the same way; an optional LLM fallback (one-word, whitelisted output) is built but has not been separately tested with a live model.
- **Over-caution:** "threatened" fires the emergency screen and drops the deposit problem; one out-of-scope word ("harassment") refuses an otherwise in-scope salary claim.
- **Hinglish** is understood but answered in English; selected-language translation for Tamil/Bengali/etc. needs an enabled LLM and may fail on provider quota. The interface itself remains localized offline.
- **Untested / unreviewed:** the owner manually confirmed Punjabi translation and a response using Gemini on their local machine; this is not part of the automated offline test suite. Live topic-classifier fallback, browser voice (mic), and lawyer review remain untested. Selected claims were checked against official India Code, Gazette and department sources. Consumer Commission limits use the value paid as consideration, not necessarily the refund due; state and transition rules may apply. Section 59 of the Industrial Relations Code covers only specified categories of money due to a worker, not every unpaid-salary claim.

6. Scale and cost

- **Cost:** offline mode makes 0 model calls (free). With an LLM, English costs 0 calls; Hindi/other-language adds at most 2 small calls per message (translate in, simplify out). I did not measure rupee cost; with a small model it should be a fraction of a rupee per conversation, to be confirmed.
- **New input:** `python3 run_story.py story.txt` (or a folder) reads any story and writes a full report, so a whole batch can be processed with no chat.
- **Adding a legal area** = 1 knowledge-base file section + a keyword list + a notice template + 4 roleplay objections + test cases. No retraining. Adding a state's rent act or labour office is a knowledge-base entry.
- **Next steps:** lawyer-reviewed knowledge base; broader evaluation of all nine selected reply languages; semantic classifier (LLM or embeddings) behind the same guard; WhatsApp front-end; log (consented) failed conversations to find the next area to cover.

7. AI tools used, and where they got it wrong

Built with AI coding assistance, with me setting scope and reviewing the outputs. **Where the AI got it wrong:** in testing, the parser read "80000 salary" as 80,000 years; I fixed the date parsing and added a regression check. An early legal draft cited two laws that had been replaced; I updated the knowledge base after checking current sources and recorded the verification limits.

Finally, a fake-model probe invented an Act and case; the output guard blocked all 7 attack variants. These failures led to narrower grounded answers, regression tests and an explicit lawyer-review step.